Data Quality Validation Report

Part 1: Data Quality Issues Found

The dataset **demand_enriched_corrupted.parquet** contains clean data before January 16, 2026 (6,079,392 rows) and corrupted data on or after that date (250,853 rows). Comparing the two windows revealed four distinct issues:

#	Issue	Rows Affected	Impact on Model
1	Negative trip_count	353 rows	Physically impossible; corrupts lag & rolling mean features
2	Extreme outliers (trip_count > 3,100)	311 rows (max = 99,999)	Inflates predicted demand by orders of magnitude
3	Duplicate rows	8,134 rows (3.2%)	Double-counts demand; skews zone baselines and rolling stats
4	Distribution shift (std ratio = 112x)	All 250,853 rows	Model trained on std=21.6; incoming std=2,433 — predictions unreliable

Part 2: Validation Approach & Graceful Degradation

Validation Functions

All checks are implemented in **validation/check_data_quality.py** via the **DataQualityValidator** class. The validator is initialized with baseline statistics derived from pre-January 16 data, then run against the incoming window:

Check	Method	Threshold	Detection Logic
Negative values	check_value_ranges()	trip_count < 0	Count rows below zero; flag as CRITICAL if any found
Extreme outliers	check_value_ranges()	> 10x baseline max (> 3,100)	Count rows exceeding 10x the historical maximum trip count
Duplicate rows	check_duplicates()	Any exact duplicate	df.duplicated().sum() > 0
Distribution shift	check_distributions()	std ratio > 5x baseline	current_std / baseline_std > 5.0

Graceful Degradation

The API must never crash due to data quality issues. Graceful degradation is implemented via **check_and_log_data_quality()** in **week3/backend/data.py**, called at startup alongside _load() and _load_model(). The function follows three principles:

1. Always continue: The entire function is wrapped in a try/except. Even if the corrupted file is missing or unreadable, the API starts normally and logs the error — it never blocks the server from coming up.

2. Always log: Every detected issue is emitted as a structured WARNING log entry with severity, type, and description. Operators monitoring pod logs will immediately see entries like: [DQ] [CRITICAL] negative_trip_count: 353 rows have negative trip_count values.

3. Serve clean data: The API continues serving from the clean Week 2 baseline (demand_enriched.parquet), not the corrupted file. The corrupted file is only read for validation and logging — it is never used to update the in-memory demand profile. This means predictions remain accurate even when new data is bad.

Part 3: Validation Strategy & Trade-offs

Where Validation Runs

Validation runs in two places: at API startup (inside data.py) and on a scheduled GitHub Actions workflow (validate-data.yml). This hybrid approach catches issues both when a new pod starts and continuously during operation.

Scheduling Decision: Hourly

The GitHub Actions workflow runs on a **cron: '0 * * * *'** schedule (every hour). This was chosen over the alternatives for the following reasons:

Frequency	Detection Lag	Cost	Verdict
Every 15 min	~15 min	High (96 runs/day)	Overkill for batch data that updates at most hourly
Every hour	~60 min	Moderate (24 runs/day)	**Selected : good balance of cost and detection speed**
Daily	Up to 24 hrs	Low (1 run/day)	Too slow for operational use
At startup only	Next deploy	Minimal	Misses corruption that occurs mid-deployment

Monitoring & Alerting

When validation fails, the workflow exits with code 1, triggering GitHub's built-in failure notification. The 'Block deployment if validation fails' step prints a banner and halts the pipeline, preventing any downstream CD steps from deploying corrupted data to GKE. Pod-level logging via Python's logging module ensures issues are visible in kubectl logs and any log aggregation system (e.g. Google Cloud Logging) attached to the cluster.

Key Trade-offs

Decision	Trade-off
Hourly schedule vs. 15-min	Saves ~72 GitHub Actions minutes/day; acceptable for batch data that updates at most once per hour
Startup validation in data.py	Adds ~2s to pod startup time; worth it for immediate operator awareness on each new deploy
std ratio vs. median shift for distribution detection	Median is robust to outliers but missed this shift (ratio 0.37). Std ratio (112x) correctly captured the variance explosion caused by injected spikes
No auto-remediation	Validation logs and blocks — it does not auto-drop bad rows. Keeps humans in the loop for data pipeline decisions; avoids silently discarding data that might be valid edge cases